1-Visão geral e contrato de negócios (SDD)

Pedro Henrique Christ
Ciência da computação
Campus Asa Norte
pedro.christ@sempreceub.com

2-Propósito do Módulo Task Analyzer

O módulo Task Analyzer tem como propósito realizar a análise de tarefas e produtividade de uma equipe ou indivíduo, processando um conjunto de tarefas de entrada e retornando métricas quantitativas que apoiem a tomada de decisão.

As principais métricas calculadas pelo módulo são:

Tempo médio de conclusão: média do tempo decorrido entre a data de início e a data de término das tarefas concluídas.

Taxa de atraso: percentual de tarefas cuja data de conclusão ultrapassou o prazo (deadline) estabelecido.

Indicadores por prioridade: agrupamento das métricas anteriores segmentado pelas prioridades das tarefas (ex.: alta, média, baixa), permitindo identificar em quais categorias há maior incidência de atrasos ou maior tempo de execução.

O módulo não realiza persistência de dados nem interface gráfica — seu escopo se restringe ao processamento analítico das informações recebidas, retornando estruturas de dados com os resultados calculados

3-Contrato Executável de Interface

Essa parte é a mais importante do documento — ela vai virar a "fonte da verdade" para o código gerado por IA na Fase 2. Copie e adapte:

Entradas (Input)

O módulo recebe uma lista de tarefas, onde cada tarefa é representada por uma estrutura de dados com os seguintes campos:

Campo	Tipo	Obrigatório	Descrição
`id`	`str`	Sim	Identificador único da tarefa
`titulo`	`str`	Sim	Nome/descrição da tarefa

`prioridade`	`str` (enum: "alta", "media", "baixa")	Sim	Nível de prioridade da tarefa
`data_inicio`	`datetime`	Sim	Data/hora de início da tarefa
`data_termino`	`datetime | None`	Não	Data/hora de conclusão (None se ainda não concluída)
`data_limite`	`datetime`	Sim	Prazo (deadline) estabelecido para a tarefa

Saídas (Output)

O módulo retorna uma estrutura de dados contendo:

Campo	Tipo	Descrição
`tempo_medio_conclusao`	`float`	Tempo médio (em horas) entre início e término das tarefas concluídas
`taxa_atraso`	`float`	Percentual (0 a 100) de tarefas concluídas após a `data_limite`
`indicadores_por_prioridade`	`dict[str, dict]`	Métricas de tempo médio e taxa de atraso agrupadas por prioridade

Tipos de Dados e Restrições de Negócio

- Apenas tarefas com `data_termino` preenchida entram no cálculo de tempo médio e taxa de atraso.
- Tarefas com `data_termino` anterior à `data_inicio` são consideradas inválidas e devem disparar exceção específica.

- A divisão para cálculo de médias deve tratar o caso de lista vazia (sem gerar `ZeroDivisionError` não tratado).
- Os valores de `prioridade` aceitos são restritos ao enum definido; valores fora do domínio devem ser rejeitados.
- O módulo deve ser **puro** (sem efeitos colaterais): não grava arquivos, não acessa banco de dados, não realiza chamadas externas.

Fase 2

Especificação de Cenários de Aceite e Test Harness

Cenários de Teste (Dado–Quando–Então)

Cenário 1 — Sucesso

- **Dado** um conjunto de tarefas válidas, com `data_inicio`, `data_termino` e `data_limite` corretamente preenchidas e prioridades dentro do domínio esperado ("alta", "media", "baixa");
- **Quando** o `TaskAnalyzer` for executado sobre esse conjunto;
- **Então** o módulo deve retornar corretamente o tempo médio de conclusão, a taxa de atraso e os indicadores agrupados por prioridade, com valores numericamente consistentes com os dados de entrada.

Cenário 2 — Exceção/Erro

- **Dado** um conjunto de tarefas contendo datas inválidas (ex.: `data_termino` anterior à `data_inicio`) ou uma lista vazia de tarefas concluídas para cálculo de médias;
- **Quando** o `TaskAnalyzer` for executado sobre esse conjunto;
- **Então** o módulo deve disparar uma exceção específica e tratada (ex.: `TarefaInvalidaError` ou `DadosInsuficientesError`), evitando falhas genéricas como `ZeroDivisionError` ou `ValueError` não tratado.

Planejamento do Test Harness

Os cenários acima serão convertidos em testes automatizados utilizando o framework **pytest** na Fase 2 do projeto. O planejamento segue três etapas:

1. **Cenários Escritos**: os cenários de aceite (Dado–Quando–Então) descritos nesta especificação servem como base funcional, escritos em linguagem natural e compreensível tanto por humanos quanto pela IA responsável pela geração de código.
2. **Testes Automatizados (pytest)**: cada cenário será traduzido em uma função de teste `pytest`, utilizando fixtures para gerar conjuntos de tarefas de entrada (válidos e inválidos) e `asserts` para validar as saídas esperadas ou o disparo correto das exceções.

3. **Validação do Código Gerado por IA**: o código produzido pelo assistente de IA na Fase 2 será considerado aceito somente se **todos** os testes automatizados passarem, garantindo que a implementação respeite fielmente o contrato executável definido na Seção 1.

Fase 3

Esta seção define as regras de governança que deverão ser fornecidas como contexto ao assistente de IA responsável pela geração do código do módulo `TaskAnalyzer`, garantindo alinhamento, consistência e qualidade técnica.

Diretrizes Arquiteturais (Obrigatórias)

- Utilizar Python 3.11+ como versão mínima obrigatória.
- Aplicar type hints (anotações de tipo) em todas as funções e métodos.
- Seguir o princípio Single Responsibility (SRP) e as convenções de código limpo definidas na PEP 8.
- Documentar formalmente todos os módulos, classes, funções e parâmetros utilizando o padrão Google Style docstrings.
- Manter funções pequenas, coesas e com um único propósito.
- Tratar exceções de forma específica, com mensagens claras e informativas.
- Utilizar o módulo padrão logging para registro estruturado de eventos.
- Garantir que o código seja legível, manutenível, com nomes de variáveis e funções descritivos.

Proibições Explícitas (Regras Restritivas)

- Não utilizar bibliotecas externas não autorizadas (somente biblioteca padrão do Python).
- Não alterar, remover ou adaptar os cenários de teste fornecidos nesta especificação.
- Não modificar a estrutura de pastas definida nesta especificação.
- Não persistir dados em arquivos ou bancos de dados.
- Não alterar a assinatura (nome e parâmetros) das funções públicas sem autorização prévia.
- Não gerar código sem testes correspondentes para as novas funcionalidades.

- Não inserir código duplicado ou desnecessário.
- Não assumir comportamentos não especificados no contrato de negócio definido na Seção 1.

.